



BKP: An R Package for Beta Kernel Process Modeling

Jiangyan Zhao

School of Statistics, ECNU

jyzhao@sfs.ecnu.edu.cn

joint work with Kunhai Qing and Jin Xu

2026.07.24

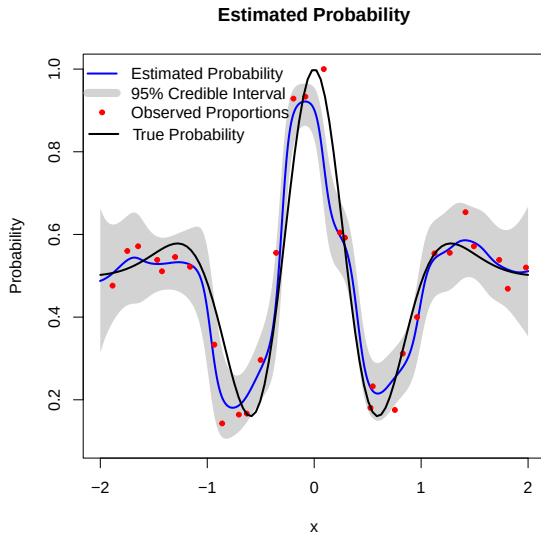


Contents

- 1 Motivation
- 2 Methodology
- 3 Package Usage
- 4 Examples



Estimating an Unknown Probability Surface



- **Observation model**

$$y_i \mid \pi(\mathbf{x}_i) \sim \text{Binomial}(m_i, \pi(\mathbf{x}_i)).$$

- **Objective**

Estimate the underlying probability surface $\pi(\mathbf{x})$ and quantify its pointwise uncertainty.



Existing Approaches and the Remaining Gap

Logistic regression

$$\pi(\mathbf{x}) = \text{logit}^{-1}(\mathbf{x}^\top \boldsymbol{\beta})$$

Fast and interpretable
Limited nonlinear flexibility

Gaussian process classification

$$\pi(\mathbf{x}) = \text{logit}^{-1}\{f(\mathbf{x})\}, \quad f(\mathbf{x}) \sim \text{GP}$$

Flexible probability surfaces
Approximate posterior inference

Motivation

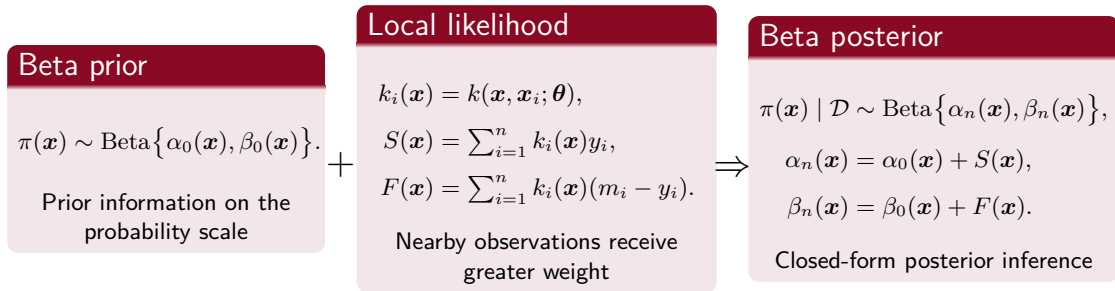
Can we retain nonlinear flexibility while performing direct, closed-form updating on the probability scale?



Contents

- 1 Motivation
- 2 Methodology
- 3 Package Usage
- 4 Examples

Beta Kernel Process: Direct Probability-Scale Updating



Key idea

Local kernel weighting + Beta-binomial conjugacy
 $\Rightarrow$ **flexible modeling with closed-form posterior inference.**



Closed-Form Posterior Summaries

Probability estimation

$$\hat{\pi}(\mathbf{x}) = \frac{\alpha_n(\mathbf{x})}{\alpha_n(\mathbf{x}) + \beta_n(\mathbf{x})}$$

Posterior mean of the unknown probability surface

Pointwise uncertainty

$$\text{Var}\{\pi(\mathbf{x}) \mid \mathcal{D}\} = \frac{\hat{\pi}(\mathbf{x})\{1 - \hat{\pi}(\mathbf{x})\}}{\alpha_n(\mathbf{x}) + \beta_n(\mathbf{x}) + 1}$$

$$\text{CI}_{1-\delta}(\mathbf{x}) = [Q_{\delta/2}(\mathbf{x}), Q_{1-\delta/2}(\mathbf{x})]$$

Credible intervals follow from Beta posterior quantiles

Count prediction

$$Y_{\text{new}} \mid \mathcal{D}, \mathbf{x}, m_{\text{new}} \sim \text{Beta-Binomial}\{m_{\text{new}}, \alpha_n(\mathbf{x}), \beta_n(\mathbf{x})\}.$$

Closed-form prediction for future binomial counts

Computational implication

Conditional on the kernel hyperparameters, posterior summaries and predictive distributions are available without numerical posterior approximation.



Prior Specification

Non-informative prior

$$\pi(\mathbf{x}) \sim \text{Beta}(1, 1).$$

Default uniform prior when no external information is available

Informative prior with fixed mean

$$\alpha_0(\mathbf{x}) = r_0 p_0, \quad \beta_0(\mathbf{x}) = r_0(1 - p_0)$$

$$\mathbb{E}\{\pi(\mathbf{x})\} = p_0, \quad \alpha_0(\mathbf{x}) + \beta_0(\mathbf{x}) = r_0$$

A fixed prior mean p_0 with precision controlled by r_0

Data-adaptive prior

$$p(\mathbf{x}) = \sum_{i=1}^n \frac{k(\mathbf{x}, \mathbf{x}_i)}{\sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)} \frac{y_i}{m_i}, \quad r(\mathbf{x}) = r_0 \sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)$$

$$\alpha_0(\mathbf{x}) = r(\mathbf{x})p(\mathbf{x}), \quad \beta_0(\mathbf{x}) = r(\mathbf{x})\{1 - p(\mathbf{x})\}.$$

The prior mean follows the local empirical proportion, while the prior precision adapts to the local kernel mass



Scaled Distance and Length-Scale Parameters

Scaled distance

$$h(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta}) = \sqrt{\sum_{j=1}^d \left(\frac{x_j - x'_j}{\theta_j} \right)^2}.$$

- Smaller θ_j : more localized borrowing and greater sensitivity along dimension j
- Larger θ_j : broader borrowing over longer distances

Length-scale structure

Isotropic: $\theta_1 = \dots = \theta_d = \theta$

Anisotropic: $\boldsymbol{\theta} = (\theta_1, \dots, \theta_d)$

An isotropic kernel uses a common length scale across dimensions, whereas an anisotropic kernel allows dimension-specific borrowing ranges

Modeling role

The length-scale parameters determine how rapidly information borrowing decreases with distance.



Kernel Functions

Implemented kernels

Gaussian: $k(h) = \exp(-h^2),$

Matérn 5/2 : $k(h) = \left(1 + \sqrt{5}h + \frac{5}{3}h^2\right) \exp(-\sqrt{5}h),$

Matérn 3/2 : $k(h) = \left(1 + \sqrt{3}h\right) \exp(-\sqrt{3}h),$

Wendland: $k(h) = (\zeta h + 1) \max\{0, 1 - h\}^\zeta, \quad \zeta = \lfloor d/2 \rfloor + 3.$

Gaussian and Matérn

Noncompact support

Every observation at a finite distance receives a positive weight, although the weight decreases with distance.

Wendland

Compact support

Observations with $h \geq 1$ receive exactly zero weight.



LOOCV-Based Length-Scale Selection

Leave-one-out quantities

- $\hat{\pi}^{-i}(\mathbf{x}_i; \boldsymbol{\theta})$: predicted probability at $\mathbf{x}_i$ after excluding observation i .
- $\tilde{\pi}_i = y_i/m_i$: observed empirical proportion.

Selection criterion

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} L(\boldsymbol{\theta}; \mathcal{D}_n), \quad L \in \{\text{BS}, \text{LL}\}.$$

Brier score

$$\text{BS}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \{ \hat{\pi}^{-i}(\mathbf{x}_i; \boldsymbol{\theta}) - \tilde{\pi}_i \}^2.$$

Log-loss

$$\text{LL}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \left[\tilde{\pi}_i \log \hat{\pi}^{-i}(\mathbf{x}_i; \boldsymbol{\theta}) + (1 - \tilde{\pi}_i) \log \{1 - \hat{\pi}^{-i}(\mathbf{x}_i; \boldsymbol{\theta})\} \right].$$

All leave-one-out predictions are obtained from one kernel matrix by setting its diagonal entries to zero.

Multi-Start Hyperparameter Optimization

Log-scale reparameterization

$$\gamma_j = \log_{10}(\theta_j), \quad \theta_j = 10^{\gamma_j}.$$

$$d_\gamma = \begin{cases} 1, & \text{isotropic,} \\ d, & \text{anisotropic.} \end{cases}$$

The transformation enforces positive length-scale parameters

Optimization workflow

- ① Generate $n_0 = 10d_\gamma$ initial values using a Latin hypercube design
- ② Run the derivative-free SBPLX algorithm from each initial value
- ③ Retain the solution with the smallest LOOCV loss

$$s^* = \operatorname{argmin}_{1 \leq s \leq n_0} L(\hat{\gamma}^{(s)}; \mathcal{D}_n), \quad \hat{\gamma} = \hat{\gamma}^{(s^*)}.$$

Why multi-start?

The LOOCV objective may be non-convex, so multiple initial values reduce sensitivity to a single local optimum.



Contents

- 1 Motivation
- 2 Methodology
- 3 Package Usage**
- 4 Examples



fit_BKP() — Data Inputs

```
fit_BKP(X, y, m, Xbounds = NULL,  
prior = "noninformative", r0 = 2, p0 = mean(y / m),  
kernel = "gaussian", isotropic = TRUE, theta = NULL,  
loss = "brier", ess = "none", n_multi_start = NULL, n_threads = 1)
```

- The first group specifies the core **data inputs**:
 - X: an $n \times d$ numeric input matrix or data frame
 - y: a length- n vector of observed success counts
 - m: a length- n vector of corresponding binomial trial counts
 - Xbounds: an optional $d \times 2$ matrix of lower and upper bounds, used to normalize X to $[0, 1]^d$ before fitting. If NULL, X is assumed to be already normalized.



fit_BKP() — Prior Specification

```
fit_BKP(X, y, m, Xbounds = NULL,  
        prior = "noninformative", r0 = 2, p0 = mean(y / m),  
        kernel = "gaussian", isotropic = TRUE, theta = NULL,  
        loss = "brier", ess = "none", n_multi_start = NULL, n_threads = 1)
```

- The second group specifies the **prior distribution**:
 - prior: selects "noninformative" (default), "fixed", or "adaptive"
 - the prior is constructed using the get_prior() function
 - r0 and p0: global prior precision and mean



fit_BKP() — Kernel Specification

```
fit_BKP(X, y, m, Xbounds = NULL,  
prior = "noninformative", r0 = 2, p0 = mean(y / m),  
kernel = "gaussian", isotropic = TRUE, theta = NULL,  
loss = "brier", ess = "none", n_multi_start = NULL, n_threads = 1)
```

- The third group specifies the **kernel and length-scale structure**:
 - kernel: "gaussian" (default), "matern52", "matern32", or "wendland"
 - isotropic: TRUE uses one shared length scale; FALSE uses dimension-specific length scales
 - theta: a positive scalar for isotropic kernels, or a scalar/length- d vector for anisotropic kernels
 - If theta = NULL, it is selected by LOOCV; otherwise, the supplied value is used and optimization is skipped



fit_BKP() — Tuning and Computation

```
fit_BKP(X, y, m, Xbounds = NULL,  
prior = "noninformative", r0 = 2, p0 = mean(y / m),  
kernel = "gaussian", isotropic = TRUE, theta = NULL,  
loss = "brier", ess = "none", n_multi_start = NULL, n_threads = 1)
```

- The final group controls **hyperparameter tuning, ESS calibration, and computation**:
 - loss: "brier" (default) or "log_loss" for LOOCV tuning
 - ess: "none" uses the standard BKP update; "shepard" calibrates the kernel-weighted data precision
 - n_multi_start: number of optimization starts; if NULL, the default is 10 for isotropic kernels and $10d$ for anisotropic kernels
 - n_threads: number of threads used in multi-start optimization; relevant only when $\theta = \text{NULL}$



fit_BKP() — Returned Object

```
BKPmodel <- fit_BKP(X, y, m, Xbounds = Xbounds)
```

- The returned BKP object stores:
 - the fitted kernel, length-scale parameters, and LOOCV results
 - the prior specification and normalized training data
 - the prior parameters α_0 and β_0
 - the posterior parameters α_n and β_n
- The fitted object supports standard S3 methods, including `summary()`, `predict()`, `plot()`, `simulate()`, and `print()`.



Contents

- 1 Motivation
- 2 Methodology
- 3 Package Usage
- 4 Examples**



General Linear Curve

- Let $x \in [-2, 2]$, and suppose the true Bernoulli probability function is given by

$$\pi_1(x) = \frac{1}{1 + e^{-3x}}. \quad (1)$$



```
# Define true success probability function
true_pi_fun <- function(x) {
  1/(1+exp(-3*x))
}

# Dataset
n <- 7
Xbounds <- matrix(c(-2,2), nrow = 1)
X <- lhs(n = n, rect = Xbounds)
true_pi <- true_pi_fun(X)
m <- sample(100, n, replace = TRUE)
y <- rbinom(n, size = m, prob = true_pi)

# Fit BKP model
BKP_model_1D_1 <- fit_BKP(X, y, m, Xbounds = Xbounds)
```



```
summary(BKP_model_1D_1)
```

```
##
##           Beta Kernel Process (BKP) Model
##
## Number of observations (n):  7
## Input dimensionality (d):   1
## Kernel type:                 (isotropic) gaussian
## Optimized kernel parameters: 0.1748
## Minimum achieved loss:      0.01165
## Loss function:               brier
## Prior type:                  noninformative
##
## Posterior predictive summary (training points):
##                               Mean Median    SD    Min    Max
## Posterior means      0.5328 0.7098 0.4079 0.0625 0.9407
## Posterior variances 0.0013 0.0011 0.0007 0.0007 0.0027
```

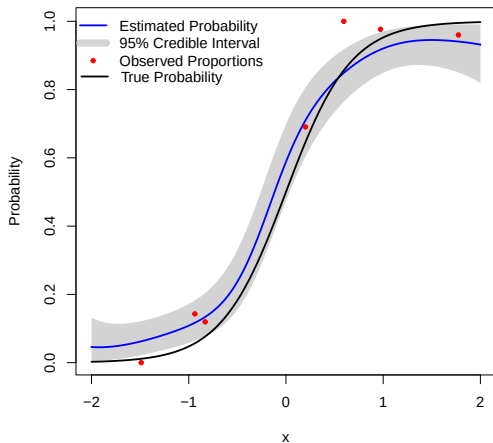


```
Xnew <- lhs(n = 10, rect = Xbounds)
predict(BKP_model_1D_1, Xnew)
```

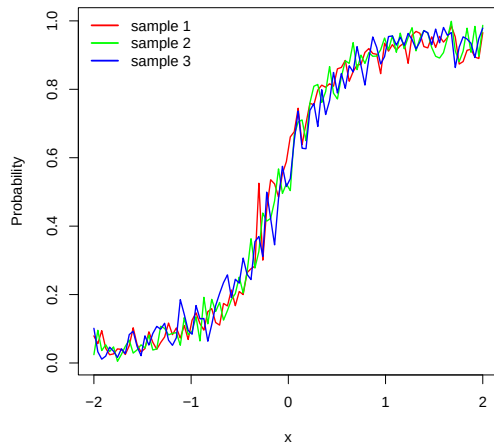
```
## Prediction results on new data (Xnew).
## Total number of prediction points: 10
##
## Preview of predictions for new data (first 6 of 10 points):
##      x      mean variance 2.5% quantile 97.5% quantile
## -0.1791 0.4512    0.0033      0.3403      0.5646
## -1.5726 0.0570    0.0007      0.0176      0.1173
##  1.4394 0.9449    0.0009      0.8714      0.9883
##  0.6069 0.8517    0.0015      0.7673      0.9198
##  1.8955 0.9365    0.0017      0.8355      0.9908
##  0.8400 0.8983    0.0011      0.8230      0.9545
##  ...
```



Estimated Probability



Simulated Probability Curves





Highly Non-linear Curve

- Define the true Bernoulli probability as

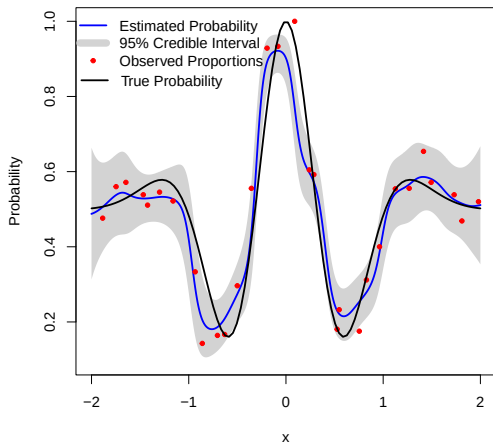
$$\pi_2(x) = \frac{1}{2} \left[1 + e^{-x^2} \cos \left(10 \frac{1 - e^{-x}}{1 + e^{-x}} \right) \right], \quad (2)$$

where $x \in [-2, 2]$.

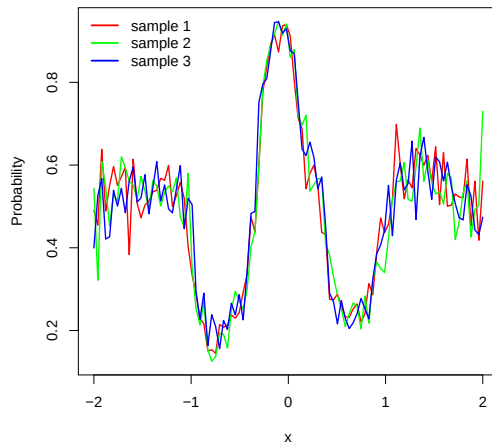
- Data size: $n = 30$



Estimated Probability



Simulated Probability Curves



2D

- Let $\mathbf{x} \in [0, 1]^2$, and define the latent surface using a re-scaled version of the Goldstein–Price function:

$$f(\mathbf{x}) = \frac{\log[\{1 + a(\mathbf{x})\}\{30 + b(\mathbf{x})\} - 8.6928]}{2.4269}, \quad \text{with}$$

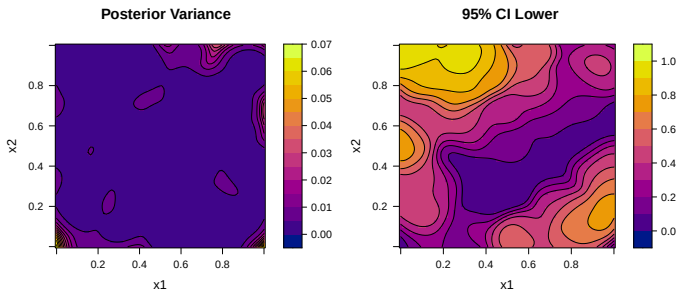
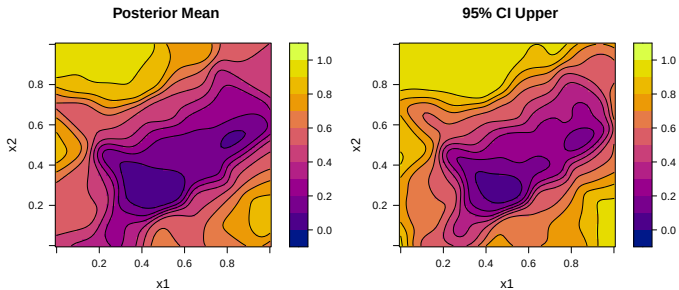
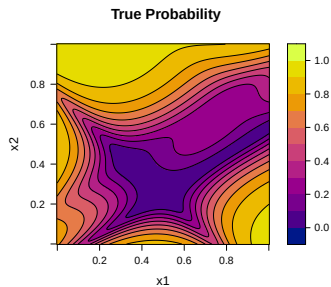
$$a(\mathbf{x}) = (4x_1 + 4x_2 - 3)^2 \times \\ \{75 - 56(x_1 + x_2) + 3(4x_1 - 2)^2 + 6(4x_1 - 2)(4x_2 - 2) + 3(4x_2 - 2)^2\},$$

$$b(\mathbf{x}) = (8x_1 - 12x_2 + 2)^2 \times \\ \{-14 - 128x_1 + 12(4x_1 - 2)^2 + 192x_2 - 36(4x_1 - 2)(4x_2 - 2) + 27(4x_2 - 2)^2\}$$

- The true Bernoulli probability surface is then defined by

$$\pi_3(\mathbf{x}) = \Phi\{f(\mathbf{x})\}, \quad (3)$$

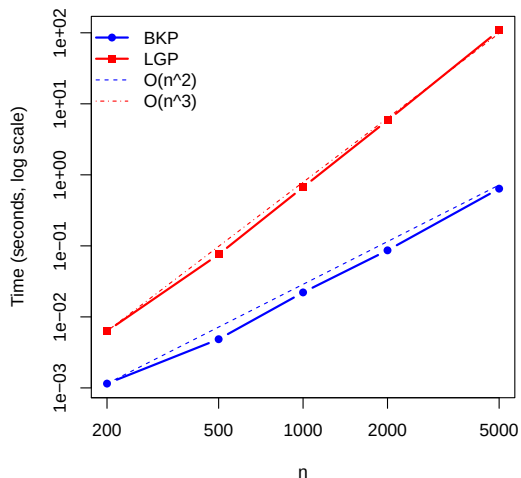
where $\Phi(\cdot)$ is the cumulative distribution function of the standard normal distribution.





Comparison of Computation Times

Fixed Hyperparameter



Optimization-based Methods

